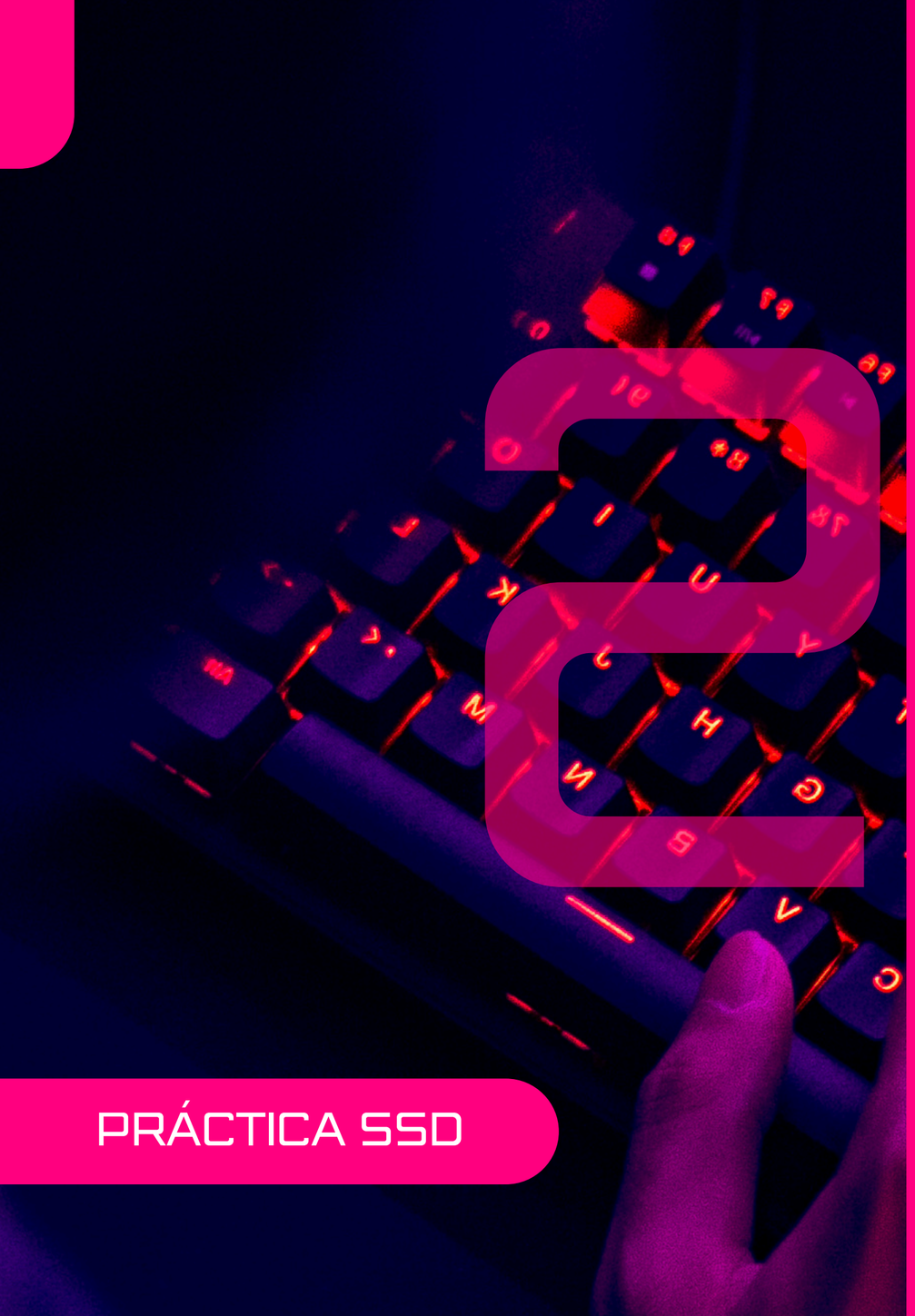



# SPEC-DRIVEN DEVELOPMENT

PRACTICA SDD

Metodología OpenSpec aplicada al diseño y evolución del videojuego Última Órbita



PRÁCTICA SSD

# ÍNDICE


## PARTE A: METODOLOGÍA SDD

Fundamentos del Spec-Driven Development, flujo OpenSpec, artefactos de especificación y evaluación crítica.

## PARTE B: EXPERIENCIA PRÁCTICA

Desarrollo del juego arcade Última Órbita, paso de MVP a supervivencia infinita y gestión formal de cambios.

## CONCLUSIONES Y MÉTRICAS

Balance metodológico, lecciones aprendidas en supervisión de la IA e indicadores de complejidad alcanzados.

MARCO DE TRABAJO

# Dirigir la IA mediante Especificaciones

### El Reto del SDD

Superar la generación caótica de código mediante contratos formales de arquitectura.

### Proyecto Práctico

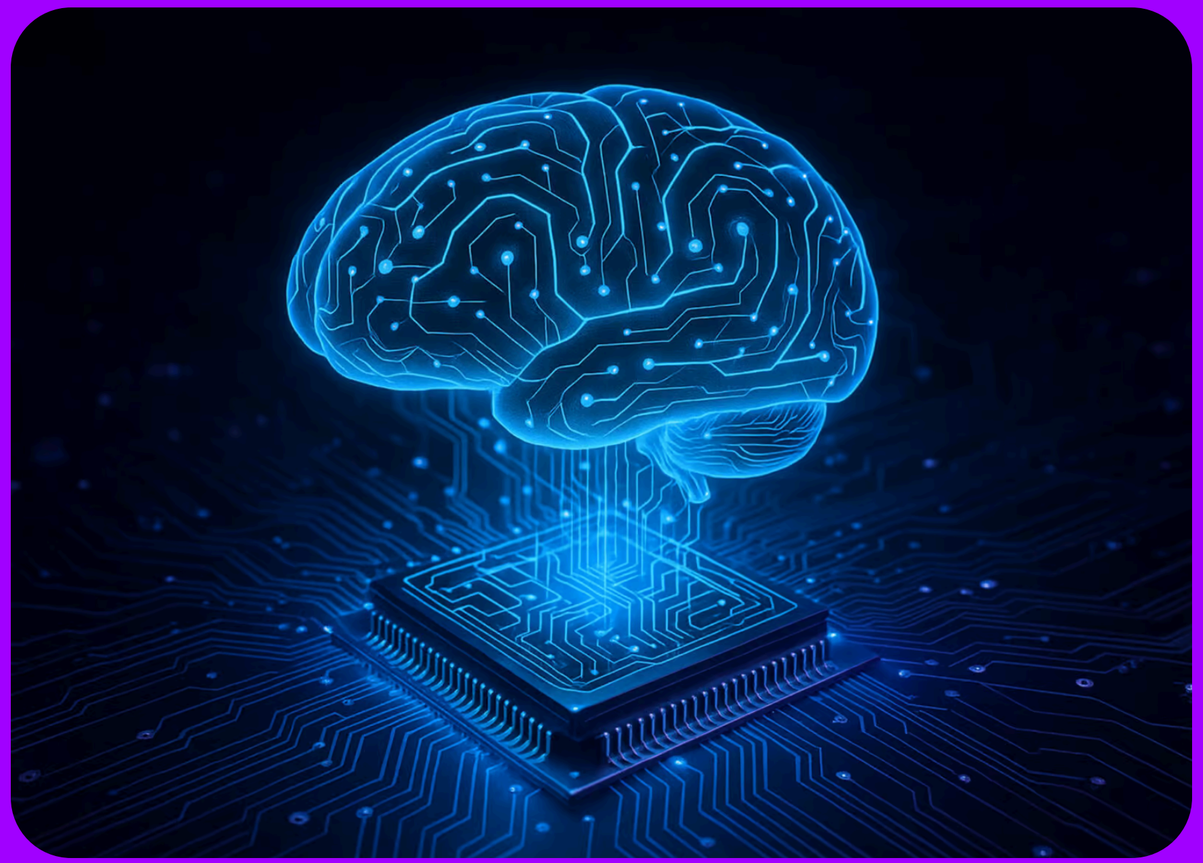
Creación del videojuego arcade **Última Órbita** con HTML5, Canvas y JavaScript.

### Gestión del Cambio

Evolución controlada desde el MVP inicial hacia un sistema de supervivencia infinita.

# Spec-Driven Development (SDD)

Spec-Driven Development (SDD) es un enfoque metodológico que antepone la definición y validación de especificaciones formales a la escritura de código. A través de un flujo gobernado que abarca desde la configuración inicial y el diseño técnico hasta la verificación final, SDD transforma los requisitos en contratos técnicos precisos, garantizando que el agente de IA ejecute exactamente la arquitectura y funcionalidades planificadas sin desviaciones ni alucinaciones.



# Flujo de Trabajo de OpenSpec

### 1. Explore (opcional)
`/opsx:explore`

Análisis previo del codebase, contexto y requisitos mediante el agente de IA.

### 2. Propose
`/opsx:propose`

Generación estructurada: proposal.md, design.md, tasks.md y carpeta specs/.

### 3. Apply
`/opsx:apply`

El agente de IA ejecuta las tareas de la especificación y genera el código real.

### 4. Verify
`/opsx:verify`

Comprobación de que el código implementado cumple con la especificación.

### 5. Sync
`/opsx:sync`

Sincronización y fusión de los delta-specs con la carpeta principal de especificaciones (specs/).

### 6. Archive
`/opsx:archive`

Consolidación del cambio en el historial a largo plazo del proyecto.

| Artefacto | Ubicación en Repositorio | Propósito Principal |
| --- | --- | --- |
| proposal.md | openspec/changes/*/proposal.md | Justificación de negocio y contexto funcional. |
| design.md | openspec/changes/*/design.md | Modelos de datos y estructura del Canvas. |
| spec.md | openspec/specs/arcade-gameplay/spec.md | Contrato funcional único y actualizado. |
| tasks.md | openspec/changes/*/tasks.md | Pasos de ejecución directos para el modelo IA. |

# Documentos Clave en OpenSpec

06

## Ventajas Principales

- **Reducción de alucinaciones:** Prompts acotados por spec.
- **Documentación viva:** El código no queda desfasado.
- **Evolución segura:** Facilita cambios de requisitos.

## Inconvenientes

- **Overhead inicial:** Trabajo previo en cambios pequeños.
- **Curva de aprendizaje:** Rigor en el flujo de carpetas.
- **Mantenimiento:** Sincronización obligatoria continua.

# Ventajas e Inconvenientes

07

# Última Órbita

## Arquitectura e Infraestructura

**Motor Gráfico Vanilla**
HTML5 Canvas 2D sin librerías externas.

**Recursos Vectoriales SVG**
Sprites y orbes optimizados.

**Zero dependencias**
Ejecución estática inmediata en el navegador.



# Desarrollo del Producto Mínimo Viable (MVP)

09

01 **Nave y Controles**
Movimiento fluido horizontal y disparo de láseres.

02 **Oleadas Finitas**
Generación estructurada por niveles definidos.

03 **Motor de Colisiones**
Detección precisa entre proyectiles y enemigos.

04 **HUD y Marcador**
Puntuación en tiempo real y contador de vidas.

PARTE B • FASE 2

# Gestión del Cambio

Tras completar el MVP, se introdujo la modificación de requisitos para transformar el juego en una experiencia de supervivencia infinita gestionada mediante OpenSpec.

CAMBIO 1

## Dificultad Dinámica

Escalado por tiempo efectivo de amenaza.

CAMBIO 2

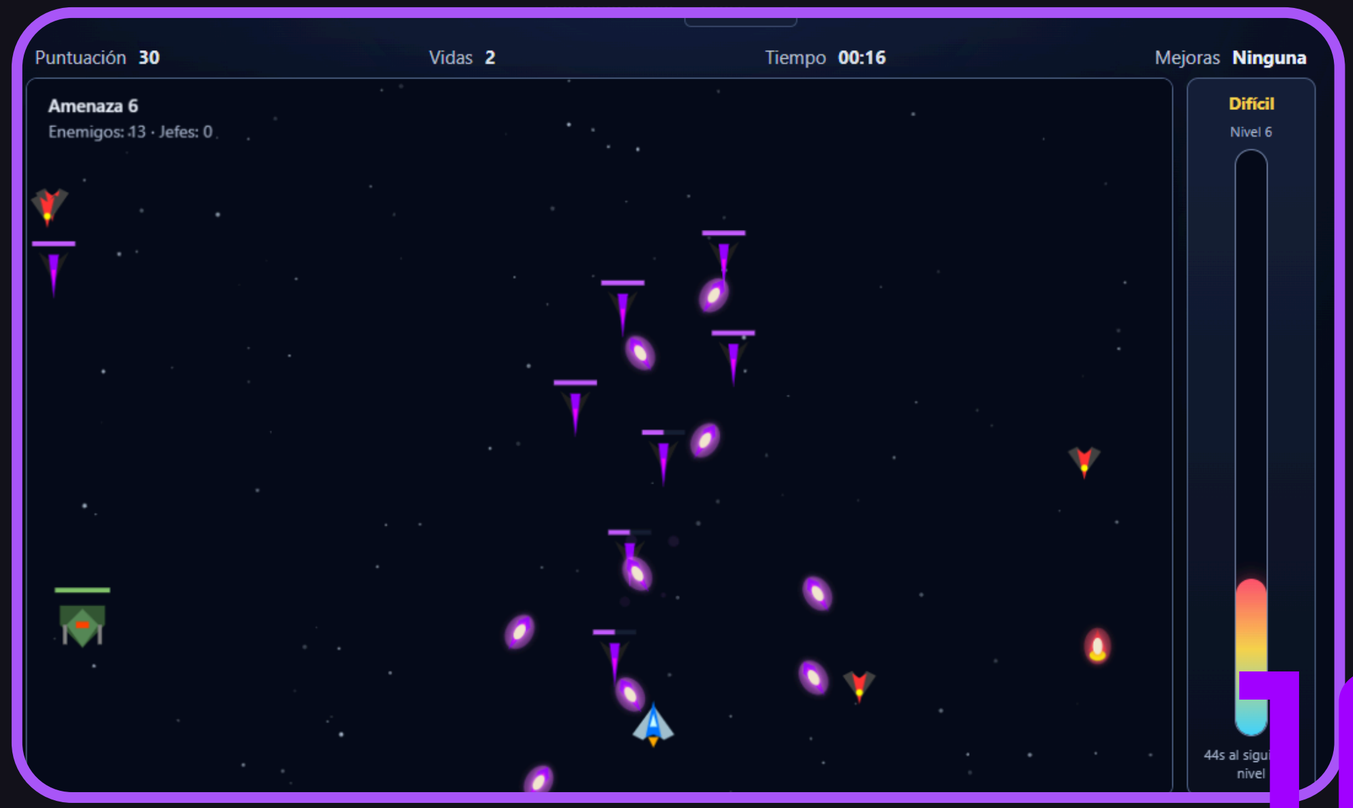
## Jefes y Escoltas

Aparición periódica de Bosses cada minuto.

CAMBIO 3

## Sistema de Orbes

6 tipos de mejoras acumulativas en partida.



# Aprendizaje de la Experiencia SDD




**01** **Control del Software**
La IA genera código velozmente, pero la especificación gobierna la dirección.

**02** **Refactorización Segura**
OpenSpec permitió remodelar el motor de oleadas sin romper el MVP.

**03** **Calidad Arquitectónica**
Código modular e inteligible desde el primer commit.

# ¡Muchas gracias!

Presentación realizada por:

Elena Hermoso Silva
Adrián Romero Maldonado
Ester Castellano Ríos